



Sheryians
Coding School

TS

TypeScript Notes

A Complete Reference Guide

01 What is TypeScript?

02 Why TypeScript?

03 Type Inference & Type Annotation

04 Primitive Data Types

05 Arrays

06 Tuples

07 Any, Unknown & Never

08 Enums

09 Union Types

10 Interfaces

11 Generics

12 Type Aliases

13 Infer Keyword

14 Utility Types

1

What is TypeScript?

TypeScript is a **superset of JavaScript** developed by Microsoft. It adds:

- Static Typing
- Better Code Completion
- Error Detection at Compile Time
- Interfaces
- Generics
- Advanced Type Features

JavaScript — no type safety

```
let age = "25";  
age = true; // Allowed ✓
```

TypeScript — type safe

```
let age: number = 25;  
age = true; // Error ✗
```

2

Why TypeScript?

❖ Catch Errors Early

```
let age: number = "25"; // Error at compile time
```

❖ Better IntelliSense

```
const user = { name: "Anubhav", age: 24 };  
user. // Editor suggests: name, age
```

❖ Easier Refactoring

Rename a property or function — TypeScript safely updates all references.

◆ **Better Maintainability**

Large projects become easier to manage with clear type contracts.

◆ **Improved Team Collaboration**

Type definitions act as self-documenting code for the entire team.

3 Type Inference & Type Annotation

Type Inference — TypeScript detects type automatically

```
let name = "Anubhav"; // inferred: string
let age = 24;          // inferred: number
```

Type Annotation — You explicitly define the type

```
let name: string = "Anubhav";
let age: number = 24;
```

Function Annotation

```
function add(a: number, b: number): number {
  return a + b;
}
```

4 Primitive Data Types

• **string**

```
let username: string = "Anubhav";
```

• **number**



```
let age: number = 24;
```

• boolean



```
let isLoggedIn: boolean = true;
```

• null



```
let value: null = null;
```

• undefined



```
let value: undefined = undefined;
```

• bigint



```
let big: bigint = 12345678901234567890n;
```

• symbol



```
let id: symbol = Symbol("id");
```

5

Arrays

Arrays store multiple values of the **same type**.

Syntax 1



```
let numbers: number[] = [1, 2, 3];
```

Syntax 2 — Generic



```
let numbers: Array<number> = [1, 2, 3];
```

String Array



```
let names: string[] = ["Anubhav", "Rahul"];
```

Mixed Array — Union



```
let data: (string | number)[] = [1, "Hello"];
```

6 Tuples

Tuple = **Fixed number** of elements with **fixed types**.



```
let user: [string, number] = ["Anubhav", 24];
```

Extended Example



```
let person: [string, number, boolean];  
person = ["Anubhav", 24, true];
```

Real World — HTTP Response



```
const response: [number, string] = [200, "Success"];
```

7 Any, Unknown & Never

Feature	any	unknown	never
Type Safety	■ No	■ Yes	■ Yes
Accepts Any Value	■ Yes	■ Yes	■ No

Can Assign Anywhere	■ Yes	■ No	■ No
Represents Impossible Val	■ No	■ No	■ Yes

any — disables type checking

```

let value: any = "Hello";
value = 123;
value = true;

```

unknown — safe, must be checked

```

let value: unknown = "Hello";
if (typeof value === "string") {
  console.log(value.toUpperCase());
}

```

never — values that never occur

```

function throwError(): never {
  throw new Error("Something went wrong");
}

function infiniteLoop(): never {
  while (true) {}
}

```

8 Enums

Enums define a **set of named constants**.

Numeric Enum

```

enum Role {
  Admin,    // 0
  User,     // 1
  Guest,    // 2
}

console.log(Role.Admin); // 0

```

String Enum

```
enum Role {  
  Admin = "ADMIN",  
  User = "USER",  
  Guest = "GUEST",  
}
```

Usage

```
let role: Role = Role.Admin;
```

9 Union Types

Allows a variable to hold **multiple possible types**.

```
let value: string | number;  
value = "Hello";  
value = 100;
```

Function Example

```
function printId(id: string | number) {  
  console.log(id);  
}
```

10 Interfaces

Interfaces define the **structure of an object**.

Basic Interface

```
interface User {  
  name: string;  
  age: number;  
}
```

Optional & Readonly Properties

```
interface User {  
  readonly id: number;    // cannot be changed  
  name: string;  
  age?: number;           // optional  
}
```

Interface Extending

```
interface Person {  
  name: string;  
}  
  
interface Student extends Person {  
  course: string;  
}
```

11 Generics

Generics make components **reusable across different types**.

Without Generics

```
function getValue(value: string): string {  
  return value;  
}
```

With Generics

```
function getValue<T>(value: T): T {  
  return value;  
}  
  
getValue<string>("Hello");  
getValue<number>(100);
```

Generic Interface


```

interface ApiResponse<T> {
  data: T;
  success: boolean;
}

const user: ApiResponse<string> = {
  data: "Anubhav",
  success: true,
};

```

12 Type Aliases

Creates **custom named types**.

```

type User = {
  name: string;
  age: number;
};

type ID = string | number;
let userId: ID = 123;

```

Interface vs Type Alias

Interface	Type Alias
Object Structure	Any Type
Extend via extends	Combine using &
Re-openable	Not Re-openable
Preferred: Objects	Preferred: Unions

13 Infer Keyword

Used inside **Conditional Types** to extract types at compile time.

Extract Return Type of a Function

```

type GetReturnType<T> =
  T extends (...args: any[]) => infer R
    ? R
    : never;

function getName() { return "Anubhav"; }

type Result = GetReturnType<typeof getName>;
// Result => string

```

Extract Array Element Type

```

type ElementType<T> =
  T extends (infer U)[]
    ? U
    : never;

type Numbers = ElementType<number[]>;
// Numbers => number

```

14 Utility Types

Built-in **helper types** provided by TypeScript to transform existing types.

◆ Partial

Makes all properties optional.

```

type PartialUser = Partial<User>;
// { name?: string; age?: number; }

```

◆ Required

Makes all properties required.

```

type RequiredUser = Required<User>;
// { name: string; age: number; }

```

◆ Readonly

Makes all properties immutable.

```
type ReadonlyUser = Readonly<User>;
```

◆ Pick

Select specific properties.

```
type UserPreview = Pick<User, "name">;  
// { name: string; }
```

◆ Omit

Remove specific properties.

```
type UserWithoutAge = Omit<User, "age">;  
// { name: string; }
```

◆ Record

Creates an object type with keys K and values T.

```
type Users = Record<string, number>;  
const scores: Users = { Anubhav: 95, Rahul: 88 };
```

◆ Exclude

Removes types from a union.

```
type Result = Exclude<string | number | boolean, boolean>;  
// string | number
```

◆ Extract

Keeps only matching types.



```
type Result = Extract<string | number | boolean, string | boolean>;  
// string | boolean
```

◆ NonNullable

Removes null and undefined.



```
type Result = NonNullable<string | null | undefined>;  
// string
```